

Computer Architecture

Processor Design: Datapath and Control

📖 Detailed Student Guide — Easy Language + Visual Diagrams

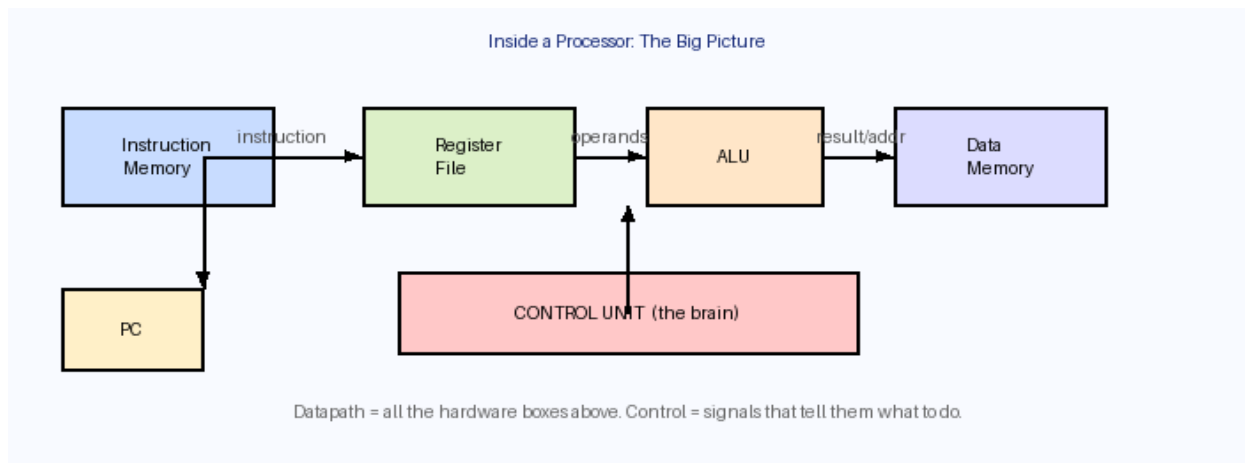


Figure 1: The big picture — a processor is made of a Datapath (the hardware) and a Control Unit (the brain)

PART 1: What is a Processor?

Understanding the big picture before the details

1.1 Why is a Processor More Than Just an ALU?

When you first learn about computers, you hear a lot about the **ALU (Arithmetic Logic Unit)** — the part that does math. But a full processor is much, much more than that.

Think of it like a restaurant kitchen. The **ALU** is just the stove. But a kitchen also needs:

- A **recipe book** (instruction memory — where the instructions are stored)
- **Shelves** to hold ingredients (registers — fast, temporary storage inside the CPU)
- A **fridge** for long-term storage (data memory — RAM)
- A **chef** who reads the recipe and decides what to do (the control unit)
- A **counter** to track which recipe step we are on (the Program Counter)

Key Insight

A processor is a Finite State Machine (FSM) — it always knows what state it is in (which instruction it is executing), and it changes state one clock tick at a time, going through instructions one by one.

1.2 The Two Main Parts: Datapath vs. Control

Every processor is divided into exactly two logical halves. Understanding this split is the most important idea in this entire lecture.

Concept	What It Is	Analogy
Datapath	All the hardware that stores, moves, or computes data. Includes registers, memories, ALU, muxes, adders, etc.	The roads and vehicles in a city
Control	The signals that tell the datapath what to do at each moment. Which register to read? Which ALU operation? Write to memory or not?	The traffic lights and road signs
Fetch	The step where the processor gets the next instruction from memory, before it can decode or execute it.	Picking up the next job card from a pile

The processor cycle always follows the same three steps, in order, for every single instruction:

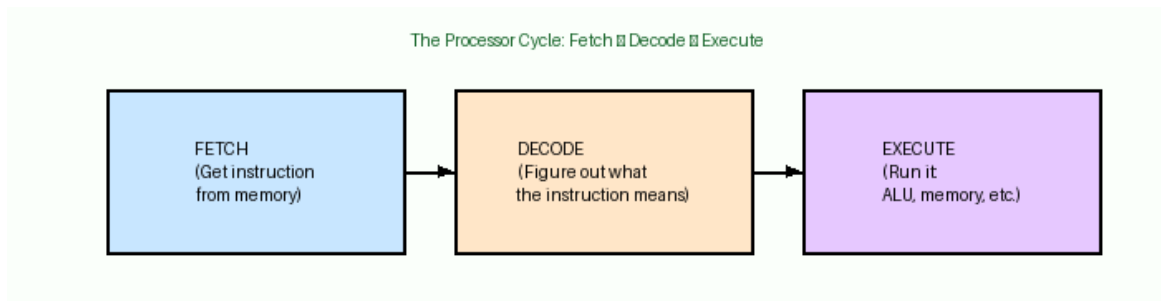


Figure 2: Fetch → Decode → Execute — the processor repeats this cycle for every instruction, forever.

PART 2: The Hardware Building Blocks

What each component does and why we need it

2.1 The Program Counter (PC)

The **Program Counter (PC)** is simply a register — a small storage element — that holds one value: the **memory address of the next instruction to execute**.

Every time the processor finishes an instruction, the PC automatically updates:

- **Normal case:** $PC = PC + 4$ (because each MIPS instruction is 4 bytes / 32 bits long, so the next instruction is 4 addresses away)
- **Branch taken:** $PC = \text{branch target address}$
- **Jump:** $PC = \text{jump target address}$

? Why + 4 and not + 1?

MIPS instructions are 32 bits = 4 bytes wide. Memory is addressed byte-by-byte. So to move to the next instruction, you add 4 bytes (not 1). If instructions were 64-bit, you would add 8.

2.2 Registers and the Register File

Registers are the **fastest storage** in a computer — even faster than RAM (they live right inside the CPU chip). Each register holds one 32-bit value.

The **Register File** is a structured collection of all the ISA (programmer-visible) registers. In MIPS:

- There are **32 registers** (numbered \$0 to \$31)
- Each register is **32 bits wide**
- The register file has **2 read ports** and **1 write port**

What does "port" mean? A port is like a door to the register file:

- **Read port:** You send in a register number (5 bits, since $\log_2(32) = 5$), and the register file sends back the 32-bit value stored there.
- **Write port:** You send in a register number, a 32-bit value, and a Write Enable (WE) signal. If $WE = 1$, the value gets written in. If $WE = 0$, nothing changes.

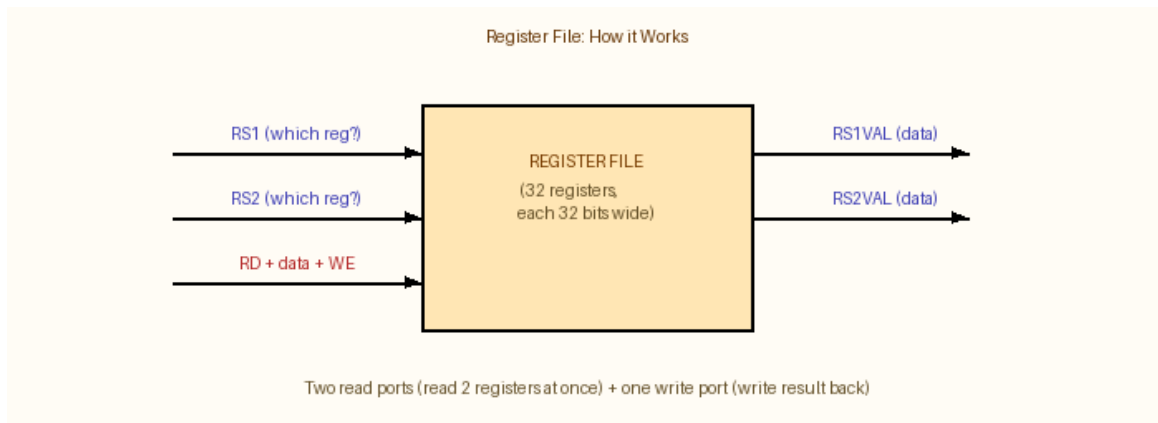


Figure 3: Register File — 2 read ports let us read 2 source registers at the same time. 1 write port stores the result.

Why do we need 2 read ports?

Consider "add \$3, \$1, \$2" — we need to read \$1 and \$2 at the same time to give both values to the ALU simultaneously. That requires 2 independent read ports. Most instructions (like R-type) need 2 source registers at once.

2.3 Memory (Instruction Memory and Data Memory)

A processor actually uses **two separate memories**:

Memory Type	What it stores	When it is accessed
Instruction Memory	The program code — the actual instructions	Every single cycle, during the Fetch stage
Data Memory	Variables, arrays, objects — the program's data (this is basically your RAM)	Only during load (lw) and store (sw) instructions

Memory is much **larger** than the register file (billions of bytes vs. 32 registers), but it is also **much slower**. This speed gap is actually the biggest performance problem in modern computers!

Memory Is Slow — This Matters A Lot

Data memory is so slow that it single-handedly determines the maximum clock speed of a single-cycle processor. The clock must wait for memory to finish before ticking. This is why we later introduce caches and pipelining.

2.4 The ALU (Arithmetic Logic Unit)

The **ALU** is the calculator of the processor. It takes two 32-bit inputs and produces a result. Operations include:

- **Addition** (add, addi, lw, sw — address calculation)
- **Subtraction** (beq — comparison is done by subtracting)
- **AND, OR, XOR** (bitwise logic operations)
- **Set Less Than** (slt — comparison)

The ALU also outputs a **Zero flag**: a 1-bit signal that is 1 if the result is zero. This is used by the **beq** (branch if equal) instruction.

2.5 Multiplexers (Muxes)

A **multiplexer (mux)** is a selector switch. It has multiple inputs and one output. A control signal (called the **select line**) chooses which input to send to the output.

Think of it as a **TV remote changing channels** — the TV (output) can show one channel at a time, selected by a button press (select signal).

In the datapath, muxes are used when the same wire needs to carry different values for different instructions. For example:

- **ALU input B mux**: For "add", input B comes from a register. For "addi", input B comes from an immediate (a constant baked into the instruction). A mux lets the control unit choose.
- **Write-back mux**: For "add", we write back the ALU result to a register. For "lw", we write back the value read from memory. A mux selects which one.
- **PC mux**: The next PC could be PC+4 (normal), a branch target, or a jump target. A mux selects the right one.

2.6 Sign Extender

MIPS instructions are 32 bits wide. Some instructions (like addi and lw) contain a **16-bit immediate value** embedded in the instruction encoding. But the ALU needs a 32-bit input.

The **sign extender** takes a 16-bit number and "stretches" it to 32 bits by copying the most significant bit (the sign bit) into the upper 16 bits. This preserves the number's value including its negative sign.

? Example of Sign Extension

If the 16-bit immediate is -5 = 1111111111111011 in binary, after sign extension to 32 bits it becomes 1111111111111111111111111111011. The value is still -5, just stored in 32 bits.

PART 3: Building the Datapath Step by Step

Adding one instruction at a time — watching the hardware grow

This is where it all comes together. The lecture builds the complete datapath by adding support for each type of instruction, one by one. Each new instruction type may require new hardware or new data paths to be added.

3.1 Step 1: Fetch (The Foundation)

Before we can execute any instruction, we must first **fetch** it — read it from instruction memory. The fetch hardware is the core of the processor:

- **PC register** — holds the current instruction address
- **Instruction Memory** — given an address, outputs the 32-bit instruction stored there
- **PC+4 Adder** — a simple adder that always computes $PC + 4$, ready to feed back as the next PC (unless a branch or jump overrides it)

📌 Every Instruction Starts Here

No matter what instruction it is — add, lw, beq, j — they all begin with Fetch. The PC sends its address to instruction memory, instruction memory sends back the instruction bits, and then Decode begins.

3.2 Step 2: add \$rd, \$rs1, \$rs2 (R-type arithmetic)

The "add" instruction reads two source registers, adds them, and writes the result to a destination register.

New hardware needed: Register File + ALU + a wire from ALU result back to register file write port.

Stage	What Happens	Hardware Used
Fetch	PC → Instruction Memory → get instruction bits	PC, Instruction Memory
Decode	Pull out rs1, rs2, rd fields from instruction bits	Instruction decoder (wiring)
Read registers	Send rs1 and rs2 numbers to Register File → get their values	Register File (2 read ports)
Execute	Send both register values to ALU → ALU adds them	ALU
Write Back	Send ALU result back to Register File → write into rd	Register File (write port), WE=1

3.3 Step 3: addi \$rt, \$rs, immediate (Immediate arithmetic)

The "addi" instruction is almost the same as "add", but the **second operand is a constant** (immediate value) embedded in the instruction, not a register.

New hardware needed:

- **Sign Extender** — to convert the 16-bit immediate into a 32-bit value
- **Mux on ALU input B** — to select between a register value (for "add") and the sign-extended immediate (for "addi")

The control signal **ALUinB** controls this mux: 0 = use register, 1 = use immediate.

Also note: the destination register for addi is **rt**, not rd (this is just how MIPS encoding works for I-type instructions).

3.4 Step 4: lw \$rt, offset(\$rs) (Load from memory)

The "lw" instruction reads a value from **data memory** and stores it into a register. The memory address is computed as: base register + sign-extended offset.

New hardware needed:

- **Data Memory** (read capability) — the actual RAM the program uses
- **Write-back mux** — to choose between writing the ALU result (for add) vs. writing the memory output (for lw) into the destination register

The control signal **Rwd (Register Write Data)** selects which value goes into the register file: ALU result or memory output.

lw is the Slowest Instruction

Load word (lw) has to pass through 5 hardware stages in sequence: instruction memory → register file (read) → ALU (address calculation) → data memory (read) → register file (write). The clock must wait for ALL of this before ticking again. This is the bottleneck of single-cycle design.

3.5 Step 5: sw \$rt, offset(\$rs) (Store to memory)

The "sw" instruction writes a register value **into** data memory. The address is computed the same way as lw.

Key differences from lw:

- **Register Write Enable (Rwe) = 0** — we are not writing to any register
- **Data Memory Write Enable (DMwe) = 1** — we are writing to memory
- **A new data path** is needed: the value from the second source register (rt) must travel to the data memory write port. For the first time, data flows "rightward" through the register file and then into memory.

3.6 Step 6: beq \$rs1, \$rs2, offset (Branch if equal)

The "beq" instruction compares two registers. If they are equal, the processor jumps to a new address (the branch target). Otherwise, execution continues normally at PC+4.

New hardware needed:

- **Zero flag from ALU** — the ALU subtracts $rs1 - rs2$. If the result is zero, they were equal. The Zero output tells us this.
- **Branch target address computation** — sign-extend the 16-bit offset, shift left by 2 (multiply by 4, to convert from word offset to byte offset), then add to PC+4 to get the target address
- **Mux on PC input** — to choose between PC+4 (normal) and the branch target. Controlled by signal **BR** (branch) AND the Zero flag. Only branch if both: the instruction IS a beq ($BR=1$) AND the registers ARE equal ($Zero=1$)

3.7 Step 7: j target (Jump)

The "j" instruction unconditionally jumps to a target address. The 26-bit jump target in the instruction is shifted left by 2 and combined with the upper 4 bits of PC+4 to form the full 32-bit jump address.

New hardware needed:

- **Jump target computation logic**
- **Another input on the PC mux** — now the PC can come from: PC+4, branch target, or jump target. A second mux (or wider mux) handles this. Controlled by signal **JP** (jump).

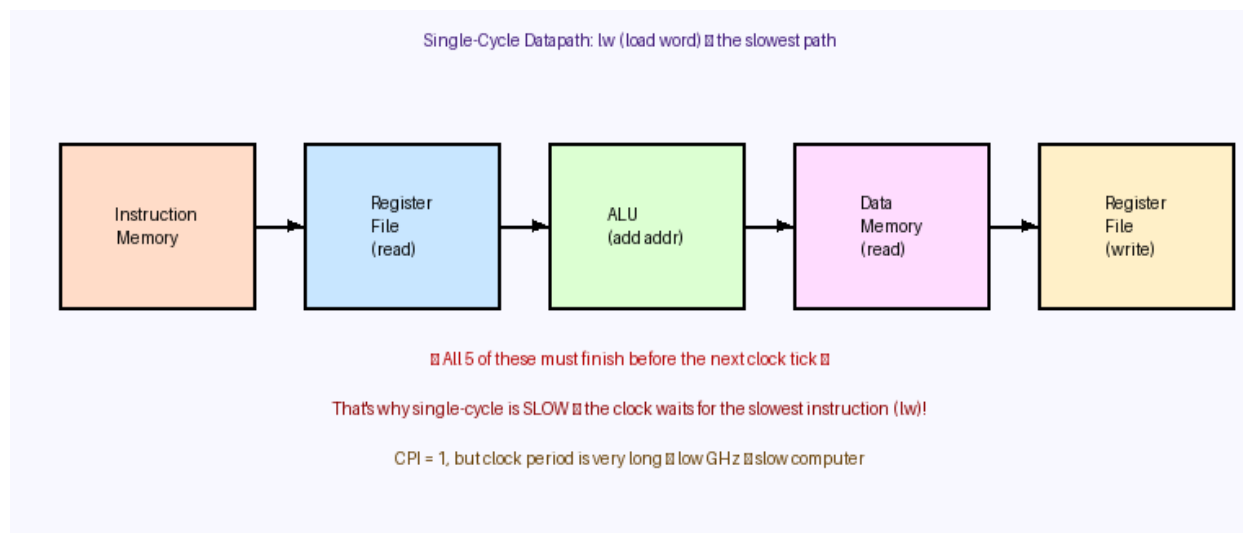


Figure 4: The lw instruction passes through 5 hardware stages in series. The clock period must accommodate all of them.

PART 4: Control — The Brain of the Processor

How the processor knows what to do for each instruction

4.1 What is Control?

You now have a datapath with registers, ALU, memory, and muxes. But there is a problem: **the hardware is passive — it does not know which instruction it is executing**. The muxes do not know which input to select. The register file does not know whether to write or not. The memory does not know whether to read or write.

That is the job of the **control unit**: to look at the instruction (specifically the **opcode** — the bits that identify what instruction it is), and produce the correct set of **control signals** that configure the datapath appropriately.

Analogy: Control is Like a Conductor

The datapath is the orchestra (instruments = hardware). The control unit is the conductor. The conductor reads the music (instruction opcode) and tells each musician exactly when and how to play. Without the conductor, the orchestra would play noise — all the instruments at once, randomly.

4.2 The 7 Main Control Signals

Our simplified MIPS datapath uses 7 key control signals. Each one is a single bit (or a few bits) that configures one part of the datapath:

Signal	Controls	0 means	1 means
Rwe	Register File Write Enable	Do NOT write to a register	Write result to destination register
Rdst	Destination register selector	Destination is rt (I-type)	Destination is rd (R-type)
ALUinB	ALU second input selector	ALU reads from a register	ALU uses sign-extended immediate
DMwe	Data Memory Write Enable	Read from memory (or ignore)	Write a value into memory
Rwd	Register Write-back Data source	Write ALU result to register	Write memory output to register
BR	Branch control	Not a branch instruction	This is a beq — check Zero flag
JP	Jump control	Not a jump instruction	This is a j — use jump target

4.3 Control Signal Table by Instruction

Here is how the control signals are set for each of the 6 instructions. "X" means **don't care** — the signal's value does not affect the result for this instruction, so we can set it to anything:

Control Signals: Same Datapath, Different Behavior

	Rwe	Rdst	ALUinB	DMwe	Rwd	BR	JP
add	Rwe=1	Rdst=rd	ALUinB=reg	DMwe=0	Rwd=ALU	BR=0	JP=0
lw	Rwe=1	Rdst=rt	ALUinB=imm	DMwe=0	Rwd=mem	BR=0	JP=0
sw	Rwe=0	Rdst=X	ALUinB=imm	DMwe=1	Rwd=X	BR=0	JP=0
beq	Rwe=0	Rdst=X	ALUinB=reg	DMwe=0	Rwd=X	BR=1	JP=0

Green = active/important | Red = disabled | Gray = don't care (X)

Figure 5: Control signal settings for each instruction. Same datapath, different control patterns.

Instruction	Rwe	Rdst	ALUinB	DMwe	Rwd	BR	JP
add	1	rd	reg	0	ALU	0	0
addi	1	rt	imm	0	ALU	0	0
lw	1	rt	imm	0	mem	0	0
sw	0	X	imm	1	X	0	0
beq	0	X	reg	0	X	1	0
j	0	X	X	0	X	0	1

4.4 How is Control Implemented in Hardware?

The control unit takes the **6-bit opcode** as input and outputs 7+ control signals. There are two main ways to build this:

Option A: ROM (Read-Only Memory) / Truth Table

Use the opcode bits as an index into a table (ROM). Each row of the table directly stores the control signal values for that opcode. Simple to understand, and easy to modify.

- **Pros:** Easy to design, easy to add new instructions
- **Cons:** Gets large for 100+ instructions. ROMs can be slow.

Option B: Combinational Logic (Gates)

Express each control signal as a Boolean logic equation of the opcode bits. Build that logic with AND, OR, NOT gates.

- **Pros:** Faster than ROM (pure logic, no memory lookup)
- **Cons:** More complex to design and modify

Real processors often use combinational logic because the control unit must be **faster than the datapath** — it cannot be the bottleneck!

⚡ Why Must Control Be Fast?

The control signals must be ready before the datapath can start executing. If the control unit is slow, the entire processor stalls waiting for it. Control hardware must propagate signals faster than it takes the ALU, memory, etc. to finish their work.

PART 5: Exceptions and Interrupts

Handling the unexpected — when the OS needs to step in

5.1 What is an Exception?

An **exception** (also called a **trap** or **fault**) is an unusual event that disrupts the normal execution of instructions. Examples include:

Exception Type	Cause	What Usually Happens
Divide by zero	A division instruction divides by 0	Program is terminated with an error
Illegal instruction	CPU encounters an unknown opcode	Program is terminated
Page fault	Program accesses memory not currently in RAM	OS loads the page from disk, then resumes
Protection fault	Program tries to access memory it does not own	OS terminates the program
Timer interrupt	A timer chip signals the OS after a time interval	OS saves state, runs a different process
I/O interrupt	A keyboard key is pressed, data arrives on network	OS handles the I/O event, then resumes
SYSCALL	Program intentionally asks OS for a service	OS performs the service (file read, etc.)

5.2 What Happens When an Exception Occurs?

When any exception occurs, the processor automatically does the following (at the hardware level):

- **Step 1 — Save the PC:** The processor saves the address of the instruction that was about to execute (or just executed) into a special register called the **EPC (Exception Program Counter)**. This lets the OS know where to return to after handling the exception.
- **Step 2 — Record the cause:** A special **Cause register** is written with a code identifying what type of exception occurred (divide by zero? page fault? etc.)
- **Step 3 — Transfer to OS:** The PC is forcibly set to the **exception handler address** — a pre-determined address where the OS exception handling code lives. The OS now runs.
- **Step 4 — OS handles it:** The OS reads EPC and Cause registers to understand the situation. It either fixes the problem (for page faults) or terminates the program.
- **Step 5 — Return (if possible):** If the OS fixed the problem, it restores the PC to the saved EPC value and returns to normal execution. The application program never even knew something happened!

❓ Exceptions Should Be Transparent

If an exception is handled and execution resumes (e.g., a page fault that was fixed), the application program should not be able to tell that an exception happened at all. It is as if it never occurred — the instruction is simply retried and this time it succeeds.

5.3 MIPS Exception Registers (Coproprocessor 0)

MIPS stores exception state in special registers on "Coproprocessor 0" — a set of registers separate from the normal \$0–\$31 registers, only accessible using privileged instructions:

Register	Number	Purpose
EPC	\$14	Stores the PC of the instruction that caused the exception (to return to later)
Cause	\$13	Stores the type of exception (SYSCALL, overflow, page fault, etc.)
BadVAddr	\$8	Stores the virtual address that caused a page or protection fault
Status/Mask	\$12	Bitmask of which exceptions are currently enabled/disabled

Privileged instructions **mfc0** and **mtc0** move values between these coprocessor registers and normal registers. User programs cannot execute these — only the OS can.

5.4 Hardware Support for Exceptions in the Datapath

The datapath must be extended to support exceptions. Key additions:

- **Coprocessor register file** (or individual registers) to hold EPC, Cause, etc.
- **PSR (Processor Status Register)**: A register that tracks whether the CPU is in user mode or privileged (kernel) mode. Some instructions and operations are only legal in kernel mode.
- **Exception detection logic**: Hardware that recognizes exception conditions (zero divisor, illegal opcode, etc.) and generates an exception signal.
- **Exception PC mux**: When an exception happens, the PC must be set to the handler address instead of the normal next-PC. A mux on the PC input handles this.

PART 6: Single-Cycle Performance — Why It Is Slow

Understanding the fundamental problem we will later solve

6.1 CPI = 1 Sounds Great... But

A **single-cycle processor** executes exactly one instruction per clock cycle. That means **CPI (Cycles Per Instruction) = 1**.

At first, CPI = 1 sounds perfect. But there is a catch:

The Performance Formula

Time per program = (# of instructions) × CPI × (time per clock cycle) CPI = 1 is good, but if the clock period is very long (few GHz), the time per program is still huge.

The problem: **the clock period must be long enough for the SLOWEST instruction** to complete from start to finish.

In our datapath, the slowest instruction is **lw (load word)** because it goes through 5 sequential hardware stages:

Stage #	Hardware	Approximate Delay
1	Instruction Memory (read)	~2 ns
2	Register File (read)	~1 ns
3	ALU (address calculation)	~2 ns
4	Data Memory (read)	~2 ns
5	Register File (write)	~1 ns
Total	Sum of all stages	~8 ns → clock can only run at ~125 MHz!

Meanwhile, an add instruction only needs stages 1, 2, 3, and 5 — it does not use data memory. It could complete in ~6 ns. But the clock still ticks every 8 ns because of lw.

Simple instructions are wasting time waiting for the clock to tick! This inefficiency is why we later introduce **pipelining** (doing multiple stages in parallel for different instructions) and **caches** (making memory faster).

Coming Up Next

The solution to single-cycle slowness is covered in the next unit: Memory Systems (caches) and Pipelining. Pipelining lets the processor work on 5 instructions simultaneously, one per pipeline stage. This is how real modern CPUs achieve 3+ GHz speeds.

PART 7: Complete Glossary of Key Terms

Quick reference for all the important words

ALU

Arithmetic Logic Unit. The part of the datapath that performs arithmetic (+, -) and logic (AND, OR) operations.

Datapath

All the hardware that stores, moves, or computes data: registers, memory, ALU, muxes, adders, etc.

Control Unit

The hardware that reads the opcode and generates control signals telling the datapath what to do.

Control Signal

A 1-bit (or few-bit) wire that configures one part of the datapath (e.g., WE=1 to enable a write).

PC (Program Counter)

A register that stores the memory address of the next instruction to execute.

Register

A tiny, very fast storage cell inside the CPU, holding one 32-bit value.

Register File

The collection of all ISA-visible registers (\$0-\$31 in MIPS), with 2 read ports and 1 write port.

Read Port

A door into the register file: send in a register number, get back its 32-bit value.

Write Port

A door into the register file: send in a register number + value + WE, and the value is stored.

Mux (Multiplexer)

A hardware switch: multiple inputs, one output, a select signal chooses which input goes to output.

Sign Extender

Hardware that widens a 16-bit immediate value to 32 bits, preserving its sign.

Instruction Memory

Memory that stores the program's instructions. Read during Fetch.

Data Memory

Memory that stores the program's data (variables, arrays). Read/written by lw and sw.

Opcode

The bits in an instruction that identify what type of instruction it is (add vs. lw vs. beq, etc.).

Immediate

A constant value baked directly into the instruction encoding, not stored in a register.

ISA

Instruction Set Architecture. The set of instructions a processor supports (e.g., MIPS, x86, ARM).

CPI

Cycles Per Instruction. For a single-cycle processor, CPI = 1 always.

Single-Cycle

A processor design where each instruction completes in exactly one clock cycle.

Exception

An unusual event (divide-by-zero, page fault, etc.) that transfers control to the OS handler.

EPC

Exception Program Counter. Stores the PC value at the time an exception occurred.

Privileged Mode

A CPU execution mode where special instructions (like mfc0) can be used; reserved for the OS.

WE (Write Enable)

A 1-bit signal. When WE = 1, a register or memory location can be written. When 0, it cannot.

Zero flag

A 1-bit output from the ALU. = 1 if the ALU result was zero. Used by beq to detect equality.

Fetch

The first stage of instruction execution: read the instruction from instruction memory using the PC.

Decode

Break the instruction bits into fields (opcode, rs, rt, rd, immediate, etc.) to figure out what to do.

Execute

Perform the actual operation: compute in ALU, access memory, and write back results.

PART 8: Exam Tips & Quick Review

What to focus on for your tests

8.1 The 5 Most Important Things to Understand

- **Datapath vs. Control:** The datapath is the hardware (passive machinery). The control unit reads the opcode and tells the datapath what to do via control signals. Never confuse the two.
- **The 3-stage cycle:** Every instruction goes through Fetch → Decode → Execute. Know what happens at each stage.
- **Trace 4 key instructions:** Be able to trace add, addi, lw, and beq through the datapath — which hardware is used, which signals are active.
- **Control signals:** Know all 7 signals (Rwe, Rdst, ALUinB, DMwe, Rwd, BR, JP) and what each one does.
- **Single-cycle bottleneck:** CPI = 1 sounds good but lw forces a long clock period. The slowest instruction limits the entire processor.

8.2 Common Exam Questions

Q: What is the difference between a register and memory?

A: Registers are inside the CPU, extremely fast, and very small in number (32 in MIPS). Memory is outside the CPU, much slower, but holds billions of bytes. Registers are used for current computation; memory for long-term data storage.

Q: Why do we need a mux on the ALU's second input?

A: Because different instructions need different second operands. "add" needs a register value; "addi", "lw", "sw" need an immediate constant. The mux lets the same ALU hardware serve both purposes by selecting which input to use based on the instruction type.

Q: Why is CPI = 1 not necessarily fast?

A: CPI = 1 means one instruction per clock cycle, but the clock period must be long enough for the slowest instruction (lw, which passes through 5 hardware stages). A long clock period means low GHz. Fast CPUs need both good CPI and high clock frequency, which is why pipelining and caching are introduced.

Q: What is the purpose of the Zero flag from the ALU?

A: The Zero flag indicates that the ALU result was zero. For the beq instruction, the ALU subtracts the two registers being compared. If they are equal, the difference is zero, and Zero = 1. The control unit uses (BR AND Zero) to decide whether to take the branch.

Q: What happens when an exception occurs?

A: (1) The current PC is saved into EPC. (2) The cause is written to the Cause register. (3) The PC is set to the pre-determined exception handler address. (4) The OS handles the exception. (5) If fixed, the OS restores the PC to EPC and resumes normal execution.

✓**End of Study Guide**

Good luck on your exams, Waseem! 🍀